An investigation into Lightweight Hand Pose estimation for Edge HCI

Brandon MacGillivray
School of Electronics, Electrical Engineering and Computer Science
Queen's University Belfast, Belfast, United Kingdom
Email: bmacgillivray01@qub.ac.uk

Abstract**—**This paper investigates lightweight 2D RGB hand pose estimation for edge-based human-computer interaction using a DRHand-inspired dual-branch architecture. The study focuses on three questions: whether a reduced 10-keypoint representation can retain sufficient utility for interaction-oriented use, whether post processing techniques such as dual-branch fusion improves prediction relative to the individual heatmap and coordinate branches, and whether pseudo-labeled real-image data can improve generalization beyond synthetic-only training. The implemented pipeline is evaluated through controlled experiments on RHD and the Hand Keypoint Dataset 26K (HK26K), together with transfer-learning runs between the two domains, comparison against a MediaPipe baseline, deployment-oriented benchmarking on Jetson Orin Nano, and qualitative analysis on selected benchmark and real-image examples. The results show that the reduced 10-keypoint model preserves most useful predictive performance while offering a modest runtime advantage, indicating that full 21-keypoint prediction is not always necessary for lightweight edge-HCI settings. Fusion improves the overall balance of results, although diagnostic analysis shows that the current heuristic remains suboptimal. Transfer learning substantially improves cross-domain performance when used for target-domain adaptation and the learned DRHand-inspired variant achieved practical embedded runtime.

Keywords**—**hand pose estimation; human-computer interaction; edge devices; lightweight deep learning; transfer learning; embedded vision

I INTRODUCTION

Point: Hand pose estimation is a relevant interface mechanism for touchless HCI. **Evidence:** Prior HCI research describes hand-based interaction as natural, expressive, and useful for pointing, manipulation, and contact-free control in settings such as AR/VR, large displays, and clinical environments [1–3]. **Explanation:** Unlike simple gesture classification, landmark-based hand pose provides a structured representation that can support richer interaction. **Link:** This makes hand-pose tracking a suitable target for practical HCI research.

Point: A significant technical challenge is achieving useful hand-pose performance under edge-device constraints. **Evidence:** Real deployment must balance prediction quality with latency, throughput, and computational cost on constrained hardware rather than desktop GPUs [4, 6, 9]. **Explanation:** A highly accurate model is not sufficient for HCI if it cannot respond quickly enough in embedded settings. **Link:** The paper therefore focuses on lightweight RGB hand-pose estimation for edge deployment.

Point: Existing lightweight methods show that real-time hand tracking is feasible, but they leave open design questions about representation, post processing techniques, and generalisation with limited training data. **Evidence:** MediaPipe Hands demonstrates efficient on-device tracking, while DRHand combines heatmap and coordinate regression in a lightweight dual-branch framework producing competitive performance results [4, 5]. **Explanation:** These methods establish a strong baseline, but certain questions emerge, such as do we need all 21 keypoints? is the fusion used in DRHand actually better than single branch inference or unnecessary computation? And in light of limited available datasets, what can be done to improve generalisation? **Link:** This motivates a targeted study of reduced keypoint prediction, fusion utility, and cross-model knowledge transfer.

Point: This paper investigates those questions through a DRHand-inspired implementation and a deployment-oriented evaluation. **Evidence:** The study compares 21-keypoint and reduced 10-keypoint prediction, evaluates fusion against the individual branches, analyses transfer between RHD and HK26K, whose annotations are used here as MediaPipe-generated pseudo-labels, compares against a MediaPipe baseline, and benchmarks runtime on NVIDIA Jetson orin nano hardware. **Explanation:** These experiments are designed to test both prediction quality and practical deployment tradeoffs. **Link:** The contribution of the paper is therefore an empirical study into current held assumptions and adaptions to dataset constraints.

II LITERATURE REVIEW

.

Point: The literature does not converge on a single architectural strategy for efficient pose estimation. **Evidence:** Existing approaches include CNN-based systems such as MediaPipe Hands [5], attention-based models [9], hybrid CNN-transformer designs such as EdgePose [14] and temporal GCN-based tracking [10]. **Explanation:** This suggests that efficient pose estimation remains an open design problem. **Link:** It is reasonable to evaluate a DRHand-inspired architecture rather than assume that the design space has already been settled.

Point: Edge deployment imposes a speed-accuracy tradeoff in pose estimation. **Evidence:** Prior work such as SPLite Hand [12] mentions the tradeoff and actively caters for it. while other papers assume desktop level compute. **Explanation:** The literature therefore shows a persistent mismatch

between models optimised for benchmark accuracy and models that are practical on embedded hardware. **Link:** This justifies treating runtime and deployment cost as core evaluation criteria

Point: Lightweight pose estimation methods typically achieve efficiency through deliberate simplification of the pipeline. **Evidence:** Reported strategies include compact backbones and lightweight attention [9], sparse or reduced-complexity computation [12] and detector-to-tracker pipelines with cropped regions of interest [5]. **Explanation:** These studies imply that efficient performance usually comes from reducing unnecessary computation across multiple stages of the model rather than from a single isolated modification. **Link:** This provides literature support for testing whether simpler prediction targets can preserve useful hand-pose performance.

Point: The choice of training data is known to affect model generalisation. **Evidence:** Prior work uses both synthetic datasets such as RHD and real datasets for hand tracking, while augmentation is often introduced to improve robustness under more realistic conditions [10, 15, 16]. **Explanation:** These data sources differ in appearance, so a model trained on one domain may not generalise reliably to another. This is especially relevant when a study relies heavily on synthetic supervision but aims to operate on real camera input. **Link:** Therefore, dataset realism and cross-domain robustness remain important considerations in lightweight hand-pose estimation.

Point: The literature leaves open whether full 21-keypoint hand representations are always necessary for lightweight edge-HCI use. **Evidence:** Existing lightweight hand-pose systems such as MediaPipe Hands [5], Attention! [9], and SPLite Hand [12] focus mainly on reducing model complexity while still predicting a full hand structure. **Explanation:** This suggests that prior work has concentrated more on compressing the model than on questioning whether the full landmark target is required for practical HCI interaction. **Link:** This directly motivates the present study's comparison between reduced 10-keypoint and standard 21-keypoint prediction.

These gaps motivate the three main questions examined in this study: whether a reduced 10-keypoint representation is sufficient for lightweight edge HCI, whether the added complexity of dual-branch fusion is justified, and whether pseudo-labeled real-image data can improve generalisation beyond synthetic-only training.

III METHOD

A Network Structure

Point: The proposed model uses a shared backbone followed by two prediction branches. **Explanation:** The input is a resized RGB hand image, and most computation is shared before the network splits into heatmap and coordinate regression paths. The same structure is used for both the standard 21-keypoint model and the reduced 10-keypoint variant, with only the output dimensionality changing.

Formally, the input to the model is an RGB image resized to 256×256, denoted by $x \in \mathbb{R}^{3\times256\times256}$. The backbone then produces a compact feature representation,

$$F = B(x).$$

A.1 The Backbone Network

Point: The backbone is designed to extract useful hand features. **Explanation:** The backbone is built from depthwise-separable convolutions, downsampling blocks, and skip-connected upsampling stages.

A.2 The Heatmap Regression Branch

Point: The heatmap branch predicts one spatial confidence map per keypoint. **Explanation:** From the shared representation, the heatmap branch predicts K output heatmaps,

$$\hat{H} = R_h(F), \qquad \hat{H} \in \mathbb{R}^{K\times64\times64},$$

where K is the number of predicted keypoints. For keypoint i, $\hat{H}_i(u,v)$ denotes the predicted confidence at spatial location (u,v) on the 64×64 heatmap grid, where u and v are the horizontal and vertical heatmap indices. Let $p_i = (x_i, y_i)$ denote the ground-truth location of keypoint i, expressed in the same heatmap coordinate system. The target heatmap for keypoint i is then defined as a 2D Gaussian centered at the true keypoint location,

$$G_i(u,v) = \exp\left(-\frac{(u-x_i)^2 + (v-y_i)^2}{2\sigma^2}\right),$$

where σ controls the spread of the Gaussian. A larger σ produces a broader peak, while a smaller σ produces a sharper target concentrated around the true landmark.

The heatmap branch is supervised using mean-squared error between the predicted heatmaps and the Gaussian target heatmaps:

$$\mathscr{L}_{hm} = \frac{1}{KHW}\sum_{i=1}^{K}\sum_{u,v}\left(\hat{H}_i(u,v) - G_i(u,v)\right)^2.$$

Here, H and W denote the heatmap height and width. This loss averages the squared difference over all keypoints and all spatial locations, encouraging each predicted heatmap to assign a high response near the true keypoint position and a low response elsewhere.

A.3 The Coordinate Regression Branch

Point: The coordinate branch predicts landmark positions directly from intermediate features. **Explanation:** Rather than producing a spatial heatmap for each keypoint, this branch regresses the landmark coordinates directly. It takes as input the intermediate 64×64 feature map produced within the heatmap pathway, allowing the coordinate branch to reuse features that already encode spatial hand structure.

The coordinate branch predicts normalised landmark coordinates,

$$\hat{P}^{coord} = R_c(F_{64}), \qquad \hat{P}^{coord} \in \mathbb{R}^{K\times2},$$

where F_{64} denotes the intermediate feature map passed from the heatmap branch to the coordinate regressor, $R_c(\cdot)$ denotes the coordinate-regression subnetwork, and K is the number

of predicted keypoints. For keypoint i, the output $\hat{p}_i^{coord} = (\hat{x}_i, \hat{y}_i)$ represents the predicted normalised 2D coordinate, with both values expressed in the range $[0,1]$.

Let $p_i = (x_i, y_i)$ denote the corresponding ground-truth normalised coordinate for keypoint i. The coordinate branch is supervised using Wing loss,

$$\mathscr{L}_{coord} = \frac{1}{2K} \sum_{i=1}^{K} \sum_{j \in \{x,y\}} \phi\left(\hat{p}_{i,j}^{coord} - p_{i,j}\right),$$

where $\hat{p}_{i,j}^{coord}$ and $p_{i,j}$ are the predicted and ground-truth values of the j-th coordinate component, and

$$\phi(\delta) = \begin{cases} w \ln\left(1 + \frac{|\delta|}{\varepsilon}\right), & |\delta| < w, \\ |\delta| - C, & \text{otherwise}, \end{cases} \qquad C = w - w \ln\left(1 + \frac{w}{\varepsilon}\right).$$

δ is the scalar prediction error for one coordinate value, w controls the range over which the loss behaves logarithmically, ε controls the curvature near zero, and C is a constant chosen so that the piecewise function is continuous at $|\delta| = w$.

Wing loss places greater emphasis on reducing small and medium localisation errors than standard L_1 or L_2 losses, while remaining less sensitive to very large errors.

B Post Processing

Point: The final pose can be produced by either branch alone or by a simple fusion rule. **Explanation:** The implementation supports three inference modes: heatmap-only, coordinate-only, and fusion. In heatmap-only mode, landmark locations are obtained by decoding the peak of each predicted heatmap. In coordinate-only mode, the direct regression output is used as the final pose. In fusion mode, the two predictions are compared joint-by-joint and a simple heuristic selects which branch to trust for each landmark.

At inference time, let $\hat{H}_i$ denote the predicted heatmap for keypoint i, with spatial dimensions $H \times W$. The heatmap-based prediction is obtained by taking the spatial argmax,

$$(u_i^*, v_i^*) = \arg\max_{u,v} \hat{H}_i(u,v),$$

where (u_i^*, v_i^*) is the location of the peak response on the discrete heatmap grid. This peak is then converted into normalized image coordinates,

$$\hat{p}_i^{hm} = \left(\frac{u_i^*}{W-1}, \frac{v_i^*}{H-1}\right).$$

The coordinate branch directly predicts the normalized coordinate $\hat{p}_i^{coord}$ for the same joint, so both outputs lie in the same $[0,1]$ coordinate space and can be compared directly.

For each joint i, the disagreement between the two branches is measured by Euclidean distance,

$$d_i = \left\| \hat{p}_i^{hm} - \hat{p}_i^{coord} \right\|_2.$$

A sample-specific threshold α is then computed from the coordinate-branch output as the median predicted bone length,

$$\alpha = \text{median}_{(a,b) \in \mathscr{E}} \left\| \hat{p}_a^{coord} - \hat{p}_b^{coord} \right\|_2,$$

where $\mathscr{E}$ denotes the set of hand-skeleton edges. α acts as a scale-adaptive tolerance: for a larger predicted hand, a larger disagreement between the two branches may still be acceptable, whereas for a smaller predicted hand, the same disagreement would be more significant.

The fused output for joint i is defined as

$$\hat{p}_i^{fused} = \begin{cases} \hat{p}_i^{hm}, & d_i < \alpha, \\ \hat{p}_i^{coord}, & \text{otherwise}. \end{cases}$$

Thus, fusion is performed independently for each joint. When the two branches agree closely, the heatmap-decoded prediction is used; otherwise, the coordinate-regression prediction is retained.

IV EXPERIMENTS

A Datasets

The Rendered Handpose Dataset (RHD) is used as the main synthetic benchmark in this study. According to the official dataset description, it provides 41,258 training samples and 2,728 testing samples. Each sample contains a rendered 320×320 RGB hand image, and 21 hand keypoints with image-space coordinates and visibility labels. the standard training/evaluation split is used.

The second dataset is HK26K (Kaggle), which is used as the more realistic image domain for transfer analysis. 18,776 training images and 7,992 validation images. The images are drawn from hand-photo and gesture datasets such as 11K Hands, 2000 Hand Gestures, and the Gesture Recognition Dataset, and include both cleaner hand-only views and more realistic photographed scenes. the annotations are generated using MediaPipe and act as pseudo-labels. the standard training/evaluation split is used.

A small real-image benchmark set, `hand_frame_small`, is also used for evaluation. This set contains 8 real RGB hand images showing basic single-hand gestures on a white background, with annotations initialized from MediaPipe predictions and then manually corrected.

B Implementation Details

The proposed network was implemented in PyTorch and trained on an NVIDIA Tesla A100 80GB GPU using the Adam optimizer. Training followed a two-stage procedure. In stage 1, only the backbone and heatmap branch were optimized with learning rate 10^{-3} and batch size 64. In stage 2, the coordinate branch was added and the model was further optimized with learning rate 10^{-4} and an effective batch size of 256, implemented as batch size 64 with gradient accumulation. Runtime benchmarking for deployment-oriented evaluation was performed separately on Jetson Orin Nano hardware.

C Differences in this implementation and DRHand

This repository is a DRHand-inspired implementation rather than a strict reproduction of the original paper. The original work was implemented in TensorFlow and trained on YouTube2D Hands and GANerated Hands, whereas the present study is implemented in PyTorch and focuses on RHD together with pseudo-labeled HK26K data. Some optimisation

details were not fully specified in the original paper, so several training values had to be reconstructed in this implementation. The reported results should therefore be interpreted as evidence about this reconstructed DRHand-style pipeline rather than as a direct numerical replication of the original study.

D Evaluation Metrics

Point: Three main pose-estimation metrics are used to evaluate prediction quality. **Explanation:** The learned models are evaluated using sum of squared error (SSE), end-point error (EPE), and probability of correct keypoint (PCK). Let $p_{si}, \hat{p}_{si} \in [0,1]^2$ denote the ground-truth and predicted normalized 2D coordinates of keypoint i in sample s, and let $v_{si} \in \{0,1\}$ denote a visibility mask, where $v_{si} = 1$ if the evaluated keypoint is visible and $v_{si} = 0$ otherwise. Thus, invisible keypoints do not contribute to the reported error values. All distances below are Euclidean distances in normalized coordinate space.

The normalised SSE is defined as

$$\mathrm{SSE} = \frac{1}{N} \sum_{s=1}^{N} \sum_{i=1}^{K} v_{si} \, \|\hat{p}_{si} - p_{si}\|_2^2 ,$$

where N is the number of samples and K is the number of evaluated keypoints. This metric sums the squared localization error over all visible evaluated keypoints in each sample and then averages that total over the dataset. Lower SSE therefore indicates lower overall pose error.

Point: End-point error is measured in a root-relative normalized form. **Explanation:** EPE is computed after subtracting a designated root joint from both the prediction and the ground truth. This removes global translation and measures how well the relative hand structure is recovered. Let r denote the root keypoint used for the current evaluation layout. In the full layout, keypoint 0 is used when available. In reduced or shared-10 evaluation, keypoint 1 is used when keypoint 0 is absent.

The normalized root-relative EPE is defined as

$$\mathrm{EPE} = \frac{\sum_{s=1}^{N} \sum_{i=1}^{K} v_{si} \, \|(\hat{p}_{si} - \hat{p}_{sr}) - (p_{si} - p_{sr})\|_2}{\sum_{s=1}^{N} \sum_{i=1}^{K} v_{si}},$$

where $\hat{p}_{sr}$ and p_{sr} are the predicted and ground-truth coordinates of the root keypoint in sample s. This metric reports the mean visible-joint distance after root alignment, so lower EPE indicates better recovery of the relative pose configuration. EPE is reported only when a supported root keypoint is present in the evaluated layout.

Point: PCK measures the proportion of visible keypoints localized within a fixed error threshold. **Explanation:** In this study, a keypoint is counted as correct when its normalized distance from the ground truth is at most 0.2. Higher PCK therefore indicates a larger fraction of acceptable landmark predictions.

$$\mathrm{PCK}_\sigma = \frac{\sum_{s=1}^{N} \sum_{i=1}^{K} v_{si} \, \mathbf{1}\left(\|\hat{p}_{si} - p_{si}\|_2 \le \sigma\right)}{\sum_{s=1}^{N} \sum_{i=1}^{K} v_{si}}, \qquad \sigma = 0.2,$$

where $\mathbf{1}(\cdot)$ is the indicator function, which returns 1 when the condition is true and 0 otherwise. Thus, PCK is the fraction of visible evaluated keypoints whose localization error falls within the accepted threshold. In this evaluation section, σ denotes the PCK threshold.

Point: Runtime is reported separately from pose accuracy. **Explanation:** Deployment-oriented evaluation includes prediction time in milliseconds per image and throughput in images per second. For the Jetson measurements, both forward-only and end-to-end timings are reported to distinguish model computation from surrounding pipeline overhead.

Point: Additional metrics are used for baseline and fusion-specific analysis. **Explanation:** For the MediaPipe baseline, detection success rate is reported because the pipeline may fail before landmark prediction. For fusion analysis, the study also reports how often each branch is selected, how strongly the two branches disagree, the typical size of the adaptive fusion threshold, and how often the heuristic agrees with an oracle branch choice computed using ground truth.

Let $m_{si} \in \{0,1\}$ denote the branch-selection decision for keypoint i in sample s, where $m_{si} = 1$ if fusion selects the heatmap prediction and $m_{si} = 0$ if fusion selects the coordinate-regression prediction. The heatmap-selection and coordinate-selection rates are then

$$\mathrm{Sel}^{hm} = \frac{1}{NK} \sum_{s=1}^{N} \sum_{i=1}^{K} m_{si}, \qquad \mathrm{Sel}^{coord} = \frac{1}{NK} \sum_{s=1}^{N} \sum_{i=1}^{K} (1 - m_{si}).$$

These rates describe how often each branch contributes the final fused output across all evaluated joints.

The reported fusion-threshold statistics are the mean and median of α_s over all evaluated samples. Intuitively, α_s reflects the predicted hand scale for that sample and determines how much disagreement between branches is tolerated before the coordinate branch is preferred.

An oracle branch label is defined for each visible evaluated keypoint by comparing the two branch errors directly against ground truth:

$$o_{si} = \mathbf{1}\left(\left\| \hat{p}_{si}^{hm} - p_{si} \right\|_2 \le \left\| \hat{p}_{si}^{coord} - p_{si} \right\|_2 \right).$$

The oracle-match rate is then

$$\mathrm{OracleMatch} = \frac{\sum_{s=1}^{N} \sum_{i=1}^{K} v_{si} \, \mathbf{1}(m_{si} = o_{si})}{\sum_{s=1}^{N} \sum_{i=1}^{K} v_{si}}.$$

This metric reports how often the fusion heuristic selected the lower-error branch on visible evaluated keypoints. A higher oracle-match rate therefore indicates that the fusion rule more often makes the locally correct decision.

V RESULTS AND ANALYSIS

A Experimental Results

A.1 Reduced Keypoint Performance

Point: The reduced 10-keypoint model retains broadly comparable performance to the 21-keypoint baseline on the shared subset. **Evidence:** In Table I, the shared-10 evaluation of `B0` gives SSE 0.1511, EPE 0.0764, and PCK 0.9101, while the

native-10 B1 model gives SSE 0.1574, EPE 0.0706, and PCK 0.9099. **Explanation:** These values are very close, indicating that reducing the landmark set does not substantially weaken performance on the subset of keypoints most relevant to the reduced representation. **Link:** This supports the claim that a smaller landmark set can remain viable for lightweight edge-HCI use.

A.2 Runtime and Deployment Cost

Point: The runtime advantage of the 10-keypoint model appears mainly in fusion mode. **Evidence:** In Table II, the forward-time gap between B0 and B1 is much larger in fusion mode (27.43 vs 24.59 ms/image) than in heatmap mode (24.28 vs 24.00 ms/image) or coordinate mode (24.08 vs 23.91 ms/image). **Explanation:** Since the single-branch modes differ only slightly, the main efficiency gain of the reduced model appears to come from lower fusion-stage overhead rather than from a large reduction in shared network cost. **Link:** This suggests that the practical speed benefit of the 10-keypoint model is tied mainly to the post-processing stage. The minor yet consistent speed gains seen with single branch results prompt further investigation.

A.3 Branch Behaviour

Point: Fusion provides the strongest overall balance of branch behaviour. **Evidence:** In Table IV, fusion gives the best or near-best overall combination of SSE and EPE for both B0 and B1, whereas the heatmap-only and coordinate-only modes are less consistent across metrics. **Explanation:** This indicates that the two branches contribute complementary strengths, so the dual-regression design offers more than simply duplicating two alternative outputs. **Link:** Accordingly, the branch-ablation results support the use of fusion as the main operating mode of the system.

A.4 Transfer and Real-Image Adaptation

Point: The experiments show that synthetic-only training does not generalize reliably to more realistic image domains, but pseudo-labeled real-image supervision provides a useful adaptation bridge. **Evidence:** In Table V, the RHD-trained model performs strongly on RHD but poorly on HK26K, while the HK26K-trained model shows the opposite pattern. After target-domain adaptation, R_TO_C and C_TO_R both improve substantially over their cross-domain starting points. This interpretation is further supported by the small real-image benchmark in Table VI, where the HK26K-oriented models, especially C_ONLY and R_TO_C, perform more strongly than the RHD-only baselines. **Explanation:** These results suggest that the main difficulty is the domain gap between synthetic training data and more realistic image conditions. Pseudo-labeled real-image data does not provide perfect supervision, but it appears to move the model closer to the real-image domain than synthetic-only training can achieve on its own. **Link:** Therefore, the transfer results are best interpreted as evidence that pseudo-labeled real-image supervision can act as a practical adaptation bridge beyond synthetic-only training.

Point: A small real-image benchmark was also included. **Explanation:** In addition to the main benchmark results on RHD and HK26K, the models were evaluated on the hand_frame_small set, which contains eight real images with annotations initialized from MediaPipe predictions and then manually corrected. These results provide a small but explicit real-image benchmark alongside the larger synthetic and pseudo-labeled datasets.

A.5 Fusion Diagnostics

Point: The current fusion heuristic is useful but clearly imperfect. **Evidence:** In Table VIII, the oracle-match rate is only about 0.48 for B0 and 0.53 for B1, even though fusion improves aggregate accuracy in the branch-ablation study. **Explanation:** This means that the heuristic often fails to choose the lower-error branch on a per-joint basis, but still helps overall because some of its correct decisions occur on more influential prediction cases. **Link:** Therefore, the current fusion rule is beneficial, but it also remains one of the clearest opportunities for future improvement.

A.6 Hyperparameter Sensitivity

Point: The baseline hyperparameter setting is a compromise rather than an obviously dominant configuration. **Evidence:** In Table IX, several alternative settings improve one metric while worsening another; for example, E1 improves EPE and PCK relative to B0, whereas W2 gives the lowest SSE, but no single configuration is uniformly best across all reported metrics. **Explanation:** This indicates that the baseline configuration is not trivially optimal, but instead reflects a practical tradeoff between competing accuracy criteria and runtime behaviour. **Link:** As a result, the hyperparameter study suggests that the selected baseline is a reasonable compromise rather than a clearly dominated operating point.

B Qualitative Results

Point: The real-image examples favour the HK26K-oriented models. **Evidence:** In the two hand_frame_small figures, C_ONLY and especially R_TO_C place the red landmarks closer to the green annotations than B0 and B1, particularly around the inner finger cluster. **Explanation:** This suggests that the real-image examples are visually closer to the HK26K domain than to the synthetic RHD domain. **Link:** The qualitative results therefore support the transfer findings on real-image adaptation.

Point: The HK26K-only model generalizes poorly to RHD. **Evidence:** In the RHD examples 00143 and 00354, C_ONLY produces landmark groups that are visibly detached from the hand. **Explanation:** This is a clear visual failure under domain shift rather than a minor local error. **Link:** The selected figures therefore reinforce the conclusion that zero-shot HK26K-to-RHD transfer is weak.

Point: Target-domain adaptation improves the transferred HK26K model on RHD. **Evidence:** In 00354, C_TO_R is visibly better aligned to the hand than C_ONLY, even though it is not the strongest prediction in the figure. **Explanation:** Fine-tuning reduces the most obvious cross-domain failures, but does not

eliminate them completely. **Link:** This supports the interpretation that transfer is useful mainly as initialization for later target-domain adaptation.

Point: `R_TO_C` is the most consistently strong learned model across the selected figures. **Evidence:** It is among the closest models to ground truth on the two real-image examples and remains stable on the two RHD examples, unlike `C_ONLY`. **Explanation:** This indicates that RHD pretraining followed by HK26K adaptation transfers more consistently across the displayed cases than HK26K-only training. **Link:** The selected figures therefore visually support the transfer asymmetry seen in the quantitative results.

Point: MediaPipe is visually strong, but the real-image comparison is not fully independent. **Evidence:** MediaPipe aligns closely with the annotations in the two `hand_frame_small` figures, but those annotations were initialized from MediaPipe predictions before manual correction. **Explanation:** This gives MediaPipe a structural advantage on that small benchmark set. **Link:** The real-image MediaPipe comparisons should therefore be treated as illustrative rather than definitive.

VI CONCLUSION AND FUTURE WORK

Point: A reduced 10-keypoint representation is sufficient for the lightweight edge-HCI setting studied here. **Evidence:** The reduced model remains close to the baseline in predictive quality while offering a runtime advantage. **Explanation:** This shows that the full 21-keypoint output is not strictly necessary for the interaction-focused scope of this study. **Link:** The first research question is answered positively.

Point: Dual-branch fusion is justified, but only as a qualified improvement. **Evidence:** Fusion gives the best overall balance across the reported results, but the diagnostic analysis shows that the current selection rule is not fully optimal. **Explanation:** The two-branch design is useful, yet the present heuristic does not extract all of its potential benefit. **Link:** The second research question is answered as a qualified yes.

Point: Pseudo-labeled real-image data improves generalisation beyond synthetic-only training, especially when used for target-domain adaptation. **Evidence:** The transfer results show that zero-shot cross-domain generalisation is weak, but fine-tuning on the target domain substantially improves performance. **Explanation:** This means pseudo-labeled real-image supervision is helpful, but mainly as an adaptation mechanism rather than as a complete replacement for target-domain training. **Link:** The third research question is also answered positively, with that limitation.

Point: Future work should redesign the architecture specifically for reduced 10-keypoint prediction. **Evidence:** The current results show that the reduced target remains effective, but the model still follows essentially the same DRHand-inspired design as the 21-keypoint version. **Explanation:** A purpose-built reduced architecture may allow further reductions in model size and latency. **Link:** The next step is therefore to test whether a smaller model can preserve the same practical accuracy in the 10-keypoint setting.

Point: Real-image evaluation should be broadened beyond the current pilot set. **Evidence:** The present study includes only a small manually annotated real-image dataset used for supplementary quantitative and qualitative comparison. **Explanation:** A larger independently annotated real-image benchmark is needed to assess practical robustness more convincingly. **Link:** Future work should therefore prioritise broader real-world evaluation.

Point: Fusion remains one of the clearest opportunities for improvement. **Evidence:** The current diagnostics show that the heuristic often fails to select the lower-error branch, and the original DRHand formulation also leaves room for stronger branch combination [4]. **Explanation:** A more principled or learned fusion strategy could improve both robustness and accuracy. **Link:** Future work should therefore investigate improved fusion design.

Point: Future work should introduce an explicit hand-detection stage to standardise the hand input before pose estimation. **Evidence:** The current learned pipeline assumes a prepared hand region, while practical systems such as MediaPipe first localise the hand and then estimate landmarks from a normalized crop. **Explanation:** A detection stage could produce more consistent hand scale, position, and framing, making the pose input distribution closer to the data seen during training and reducing reliance on aggressive spatial augmentation. **Link:** Future work should therefore evaluate whether detector-based input normalisation improves robustness and simplifies the pose-learning stage.

TABLES

TABLE I: Main quantitative comparison between the 21-keypoint baseline and the reduced 10-keypoint model on RHD. Lower SSE and EPE are better; higher PCK@0.2 is better. Values are mean ± std over five seeds.

Model	Eval set	Train joints	SSE ↓	EPE ↓	PCK@0.2 ↑
B0 baseline	native-21	21	0.3098 ± 0.0236	0.0779 ± 0.0054	0.9112 ± 0.0071
B0 baseline	shared-10	21	0.1511 ± 0.0120	0.0764 ± 0.0039	0.9101 ± 0.0071
B1 reduced	native-10	10	0.1574 ± 0.0226	0.0706 ± 0.0054	0.9099 ± 0.0046

TABLE II: Edge-device latency summary for the main models. Forward time measures the prediction stage only, whereas end-to-end time includes image read, preprocessing, host-to-device transfer, prediction, and JSON write. Values are mean ± std over five seeds.

Model	Mode	Forward ms/img ↓	End-to-end ms/img ↓	Images/s ↑
B0	fusion	27.43 ± 0.45	36.81 ± 1.33	27.20 ± 0.97
B0	heatmap	24.28 ± 0.35	33.19 ± 0.56	30.13 ± 0.50
B0	coord	24.08 ± 0.44	33.17 ± 0.86	30.16 ± 0.77
B1	fusion	24.59 ± 0.61	33.31 ± 0.71	30.03 ± 0.64
B1	heatmap	24.00 ± 0.34	32.64 ± 0.46	30.64 ± 0.43
B1	coord	23.91 ± 0.29	32.50 ± 0.34	30.77 ± 0.32

TABLE III: Detailed latency breakdown for the main fused models. These metrics are available from the benchmark pipeline and help localize where runtime is spent. Values are mean ± std over five seeds.

Model	Read ms	Prep ms	H2D ms	Forward ms	Write ms	E2E ms
B0 fusion	4.89 ± 0.64	2.80 ± 0.13	0.32 ± 0.02	27.43 ± 0.45	0.67 ± 0.09	36.81 ± 1.33
B1 fusion	4.49 ± 0.08	2.74 ± 0.05	0.31 ± 0.01	24.59 ± 0.61	0.52 ± 0.02	33.31 ± 0.71

TABLE IV: Branch ablation for the dual-regression design. Fusion is compared against the heatmap-only and coordinate-only outputs using the same trained checkpoints. Values are mean ± std over five seeds.

Model	Eval set	Mode	SSE ↓	EPE ↓	PCK@0.2 ↑
B0	native-21	fusion	0.3098 ± 0.0236	0.0779 ± 0.0054	0.9112 ± 0.0071
B0	native-21	heatmap	0.3139 ± 0.0684	0.0989 ± 0.0335	0.9374 ± 0.0066
B0	native-21	coord	0.3134 ± 0.0229	0.0788 ± 0.0062	0.9113 ± 0.0071
B1	native-10	fusion	0.1574 ± 0.0226	0.0706 ± 0.0054	0.9099 ± 0.0046
B1	native-10	heatmap	0.2213 ± 0.0880	0.0991 ± 0.0288	0.9022 ± 0.0393
B1	native-10	coord	0.1599 ± 0.0218	0.0715 ± 0.0070	0.9102 ± 0.0048

TABLE V: Transfer-learning results for the four training sequences evaluated on both RHD and HK26K. Where C... refers to the COCO annotation of HK26K and is used as a shorthand. Values are mean ± std over five seeds, and fusion-mode evaluation is shown.

Training sequence	Exp.	RHD SSE	RHD EPE	RHD PCK	HK26K SSE	HK26K EPE	HK26K PCK
RHD scratch	`R_ONLY`	0.3091 ± 0.0269	0.0764 ± 0.0030	0.9143 ± 0.0097	3.7488 ± 0.7985	0.3365 ± 0.0075	0.3719 ± 0.0434
HK26K scratch	`C_ONLY`	1.4194 ± 0.2619	0.1609 ± 0.0207	0.3397 ± 0.0718	0.0764 ± 0.0016	0.0612 ± 0.0049	0.9834 ± 0.0007
HK26K → RHD	`C_TO_R`	0.4504 ± 0.0162	0.1020 ± 0.0039	0.8190 ± 0.0114	1.6426 ± 0.2293	0.2678 ± 0.0114	0.5320 ± 0.0164
RHD → HK26K	`R_TO_C`	0.5699 ± 0.0592	0.1276 ± 0.0149	0.7690 ± 0.0221	0.0980 ± 0.0064	0.0840 ± 0.0018	0.9837 ± 0.0007

TABLE VI: Real-image benchmark evaluation on the `hand_frame_small` set ($n = 8$ images). The annotations were initialized from MediaPipe predictions and then manually corrected. Learned-model rows are mean $\pm$ std over five seeds; MediaPipe is a single run. All learned-model results use fusion-mode evaluation.

Method	Eval protocol	SSE ↓	EPE ↓	PCK@0.2 ↑	Detect. succ.
B0	shared-10	0.3345 ± 0.0221	0.2815 ± 0.0129	0.7075 ± 0.0143	n/a
B1	native-10	0.4350 ± 0.1153	0.2752 ± 0.0307	0.6475 ± 0.0555	n/a
R_ONLY	shared-10	0.3595 ± 0.0475	0.2706 ± 0.0214	0.7050 ± 0.0338	n/a
C_ONLY	shared-10	0.1215 ± 0.0434	0.1155 ± 0.0286	0.9200 ± 0.0647	n/a
C_TO_R	shared-10	0.3644 ± 0.0360	0.2481 ± 0.0090	0.7075 ± 0.0301	n/a
R_TO_C	shared-10	0.1209 ± 0.0236	0.1043 ± 0.0151	0.9425 ± 0.0301	n/a
MediaPipe	native-21	0.0012	0.0018	1.0000	1.0000
MediaPipe	shared-10	0.0005	0.0017	1.0000	1.0000

TABLE VII: Comparison between the MediaPipe Hand Landmarker baseline and the learned DRHand variants on RHD. The MediaPipe rows are single evaluation runs using `-hand auto`, whereas the DRHand rows are mean $\pm$ std over five seeds. Detection success is shown only for the MediaPipe runs because the learned-model evaluations do not use the same detection stage.

Method	Eval set	SSE	EPE	PCK@0.2	Detect. succ.
MediaPipe	native-21	1.7049	0.0833	0.7707	0.8933
MediaPipe	shared-10	0.8599	0.1378	0.7635	0.8933
`B0` fusion	native-21	0.3098 ± 0.0236	0.0779 ± 0.0054	0.9112 ± 0.0071	n/a
`B0` fusion	shared-10	0.1511 ± 0.0120	0.0764 ± 0.0039	0.9101 ± 0.0071	n/a
`B1` fusion	native-10	0.1574 ± 0.0226	0.0706 ± 0.0054	0.9099 ± 0.0046	n/a

TABLE VIII: Fusion diagnostics for the dual-regression post-processing rule. The oracle-match rate measures how often the final fusion choice matches the lower-error branch for visible joints. Values are mean $\pm$ std over five seeds. The table includes all currently available diagnostic exports, including a shared-10 row for `B0`.

Model	Eval set	HM sel.	Coord sel.	α mean	Disagr. mean	Oracle match
B0	native-21	0.426 ± 0.025	0.574 ± 0.025	0.065 ± 0.007	0.163 ± 0.022	0.481 ± 0.012
B0	shared-10	0.407 ± 0.024	0.593 ± 0.024	0.065 ± 0.007	0.162 ± 0.018	0.474 ± 0.011
B1	native-10	0.501 ± 0.053	0.499 ± 0.053	0.084 ± 0.003	0.171 ± 0.018	0.529 ± 0.037

TABLE IX: Hyperparameter ablation relative to the baseline `B0` configuration, using native-21 fused accuracy on RHD and fused end-to-end benchmark latency. Values are mean $\pm$ std over five seeds.

Exp	Change vs. B0	SSE ↓	EPE ↓	PCK@0.2 ↑	E2E ms ↓
B0	baseline	0.3098 ± 0.0236	0.0779 ± 0.0054	0.9112 ± 0.0071	36.81 ± 1.33
L1	$\lambda_{hm} = 1.5$, $\lambda_{coord} = 0.5$	0.3583 ± 0.0534	0.0725 ± 0.0096	0.9081 ± 0.0092	35.77 ± 0.45
L2	$\lambda_{hm} = 0.5$, $\lambda_{coord} = 1.5$	0.3073 ± 0.0260	0.0812 ± 0.0022	0.9115 ± 0.0095	35.75 ± 0.49
S1	$\sigma = 1.5$	0.3252 ± 0.0381	0.0707 ± 0.0055	0.9116 ± 0.0086	35.72 ± 0.25
S2	$\sigma = 3.0$	0.3137 ± 0.0488	0.0833 ± 0.0058	0.9177 ± 0.0081	36.48 ± 0.53
W1	$w = 5.0$	0.3213 ± 0.0384	0.0737 ± 0.0058	0.9150 ± 0.0099	36.21 ± 0.39
W2	$w = 15.0$	0.2979 ± 0.0441	0.0750 ± 0.0031	0.9140 ± 0.0124	35.91 ± 0.43
E1	$\varepsilon = 1.0$	0.2994 ± 0.0591	0.0695 ± 0.0011	0.9175 ± 0.0113	35.84 ± 0.23
E2	$\varepsilon = 4.0$	0.3106 ± 0.0106	0.0873 ± 0.0056	0.9110 ± 0.0067	36.13 ± 0.52

QUALITATIVE FIGURES

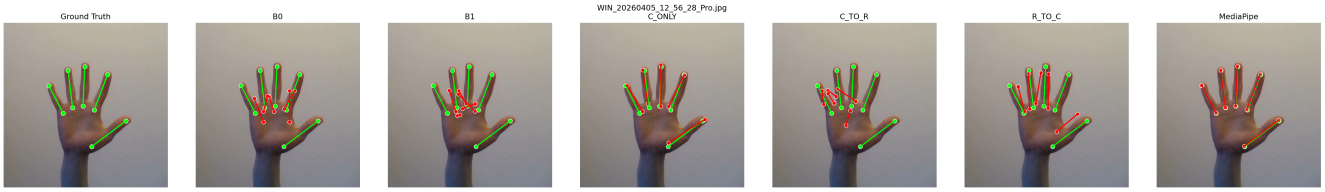

Figure 1: Qualitative comparison on hand_frame_small example 1.

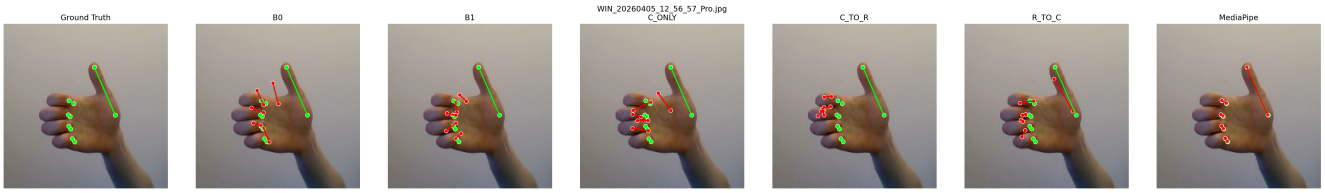

Figure 2: Qualitative comparison on hand_frame_small example 6.

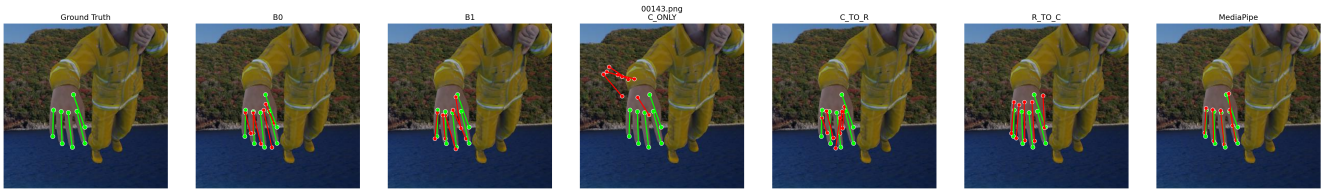

Figure 3: Qualitative comparison on RHD evaluation example 00143.

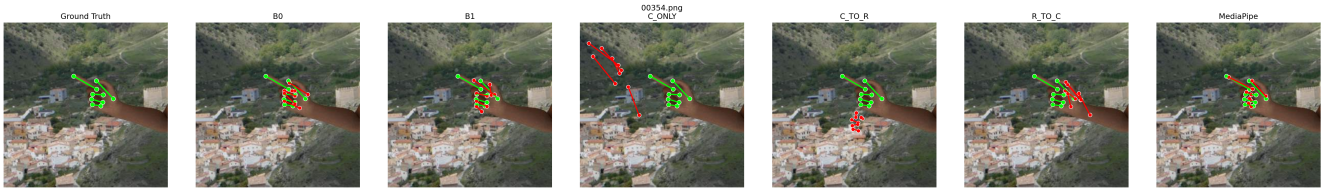

Figure 4: Qualitative comparison on RHD evaluation example 00354.

REFERENCES

[1] T. Vuletic, A. Duffy, L. Hay, C. McTeague, and G. Campbell, "Systematic literature review of hand gestures used in human-computer interaction interfaces," *Int. J. Human-Computer Studies*, vol. 129, pp. 47–65, 2019.

[2] J. P. Wachs, M. Kolsch, H. Stern, and Y. Edan, "Vision-based hand-gesture applications," *Communications of the ACM*, vol. 54, no. 2, pp. 60–71, 2011.

[3] Y. Li, J. Huang, F. Tian, H.-A. Wang, and G.-Z. Dai, "Gesture interaction in virtual reality," *Virtual Reality & Intelligent Hardware*, vol. 1, no. 1, pp. 84–112, 2019.

[4] D. Wei, S. An, X. Zhang, J. Tian, K. A. Tsintotas, A. Gasteratos, and H. Zhu, "Dual regression for efficient hand pose estimation," in *Proc. IEEE Int. Conf. Robotics and Automation*, 2022.

[5] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe hands: On-device real-time hand tracking," *arXiv preprint arXiv:2006.10214*, 2020.

[6] Y. Wang, B. Zhang, and C. Peng, "SRHandNet: Real-time 2D hand pose estimation with simultaneous region localization," *IEEE Trans. Image Processing*, vol. 29, pp. 2977–2986, 2019.

[7] Z.-H. Feng, J. Kittler, M. Awais, P. Huber, and X.-J. Wu, "Wing loss for robust facial landmark localisation with convolutional neural networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2018, pp. 2235–2245.

[8] S. An, X. Zhang, D. Wei, H. Zhu, J. Yang, and K. A. Tsintotas, "FastHand: Fast monocular hand pose estimation on embedded systems," *Journal of Systems Architecture*, vol. 122, p. 102361, 2022.

[9] N. Santavas, I. Kansizoglou, L. Bampis, E. Karakasis, and A. Gasteratos, "Attention! A lightweight 2D hand pose estimation approach," *IEEE Sensors Journal*, vol. 21, no. 10, pp. 11488–11501, 2021.

[10] Y. Yin, C. McCarthy, and D. Rezazadegan, "Real-time lightweight 3D hand-object pose estimation using temporal graph convolution networks," in *AI 2024: Advances in Artificial Intelligence*, LNCS 15443, 2024, pp. 243–255.

[11] A. Prakash, R. Tu, M. Chang, and S. Gupta, "3D hand pose estimation in everyday egocentric images," in *Proc. ECCV*, 2024.

[12] K. H. Yeh, T. W. Hsu, and M. Sun, "SPLite Hand: Sparsity-aware lightweight 3D hand pose estimation," in *Proc. AICCC 2025*, Tokyo, Japan, 2025, pp. 1–10.

[13] E. Williams-Linera and J. M. Ramirez-Cortes, "Stereo vision system based on the NVIDIA Jetson Nano for real-time evaluation of yoga poses," in *Proc. IEEE*, 2024, pp. 1–7.

[14] L. Zhang et al., "EdgePose: Real-time human pose estimation scheme for industrial scenes," *IEEE Access*, vol. 12, pp. 156702–156716, 2024.

[15] Y. Zhang et al., "A lightweight hand attitude estimation method based on GCN feature enhancement," *Electronics*, vol. 13, no. 10, p. 1952, 2024.

[16] Z. Zhang, S. Xie, M. Chen, and H. Zhu, "HandAugment: A simple data augmentation method for depth-based 3D hand pose estimation," *arXiv preprint arXiv:2001.00702*, 2020.

[17] I. B. Toft, J. P. B. Nielsen, N. K. Poulsen, and A. L. Laursen, "EfficientPose: Scalable single-person pose estimation," *arXiv preprint arXiv:2008.09062*, 2020.

[18] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," *arXiv preprint arXiv:2006.10204*, 2020.